



CSE 711 - Compilers

Lecture 2 (Part 2)

Course Instructor: Ms. Samia Muntaha
Department of Computer Science and Engineering
University of Chittagong

[Researchgate Profile](#)

Objectives

- ▶ Simulation of DFA and NFA
- ▶ Converting RE to NFA
- ▶ Converting NFA to DFA
- ▶ Converting DFA to RE

Finite State Automata (FSAs)

- ▶ **AKA “Finite State Machines”, “Finite Automata”, “FA”**
- ▶ A *recognizer* for a language is a program that takes as input a string x and answers “yes” if x is a sentence of the language and “no” otherwise.
 - ▶ The regular expression is compiled into a recognizer by constructing a generalized transition diagram called a finite automaton.
- ▶ One start state
- ▶ Many final states
- ▶ Each state is labeled with a state name
- ▶ Directed edges, labeled with symbols
- ▶ Two types
 - ▶ Deterministic (DFA)
 - ▶ Non-deterministic (NFA)

Simulation of a Finite Automata (FA)

```
s ← s0
c ← nextchar;
while c ≠ eof do
    s ← move(s, c);
    c ← nextchar;
end;
if s is in F then return "yes"
else return "no"
```

**DFA
simulation**

```
S ←  $\epsilon$ -closure({s0})
c ← nextchar;
while c ≠ eof do
    S ←  $\epsilon$ -closure(move(S, c));
    c ← nextchar;
end;
if  $S \cap F \neq \emptyset$  then return "yes"
else return "no"
```

**NFA
simulation**



Nondeterministic Finite Automata

A nondeterministic finite automaton (NFA) is a mathematical model that consists of

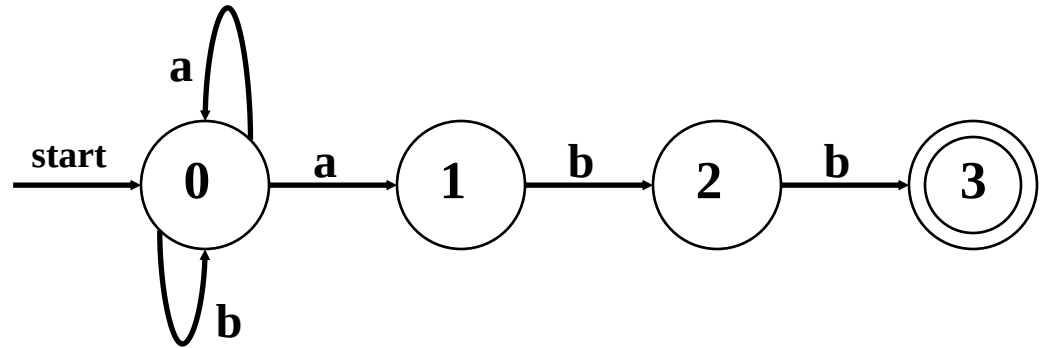
1. A set of states S
2. A set of input symbols S
3. A transition function that maps state/symbol pairs to a set of states
1. A special state s_0 called the start state
2. A set of states F (subset of S) of final states

INPUT: string

OUTPUT: yes or no

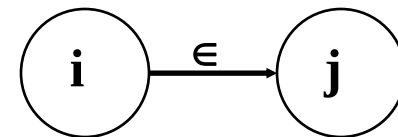
Example – NFA : $(a \mid b)^*abb$

$S = \{ 0, 1, 2, 3 \}$
 $s_0 = 0$
 $F = \{ 3 \}$
 $\Sigma = \{ a, b \}$



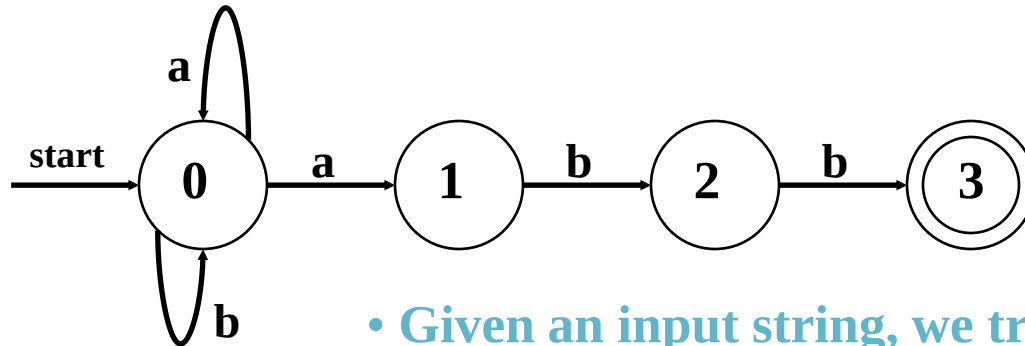
	input	
	a	b
s	0	{ 0, 1 }
t		{ 0 }
a	1	--
t	2	{ 2 }
e		{ 3 }

ϵ (null) moves possible



Switch state but do not use any input symbol

How Does An NFA Work ?



- Given an input string, we trace moves
- If no more input & in final state, ACCEPT

EXAMPLE:

Input: ababb

$move(0, a) = 1$
 $move(1, b) = 2$
 $move(2, a) = ?$ (undefined)

REJECT !

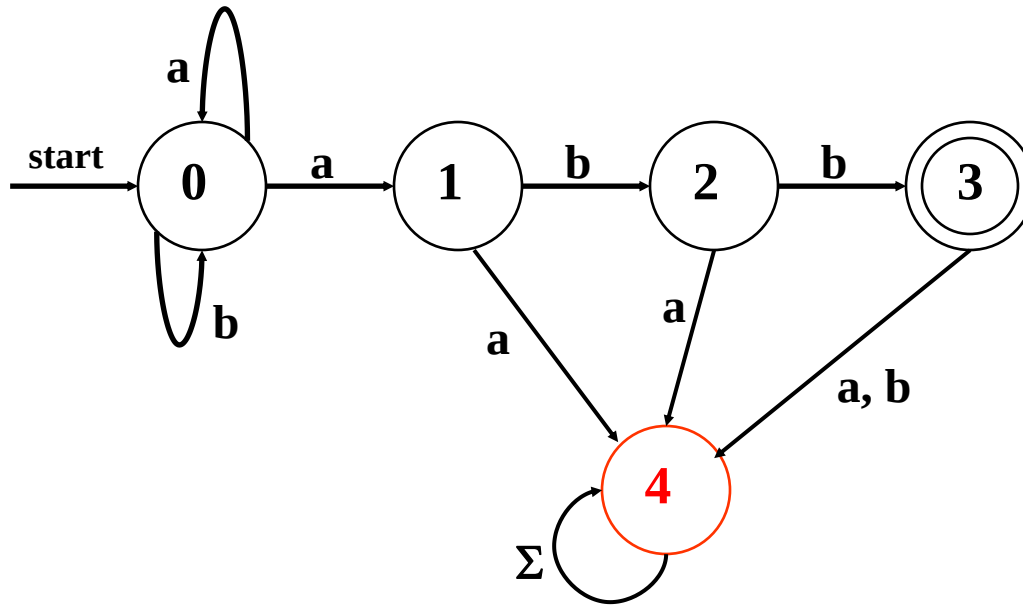
-OR-

$move(0, a) = 0$
 $move(0, b) = 0$
 $move(0, a) = 1$
 $move(1, b) = 2$
 $move(2, b) = 3$

ACCEPT !

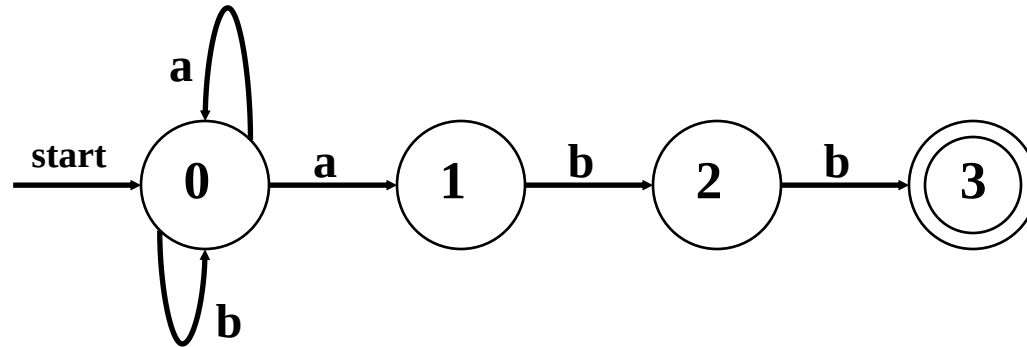
Handling Undefined Transitions

We can handle undefined transitions by defining one more state, a “death” state, and transitioning all previously undefined transition to this **death state**.



Other Concepts

Not all paths may result in acceptance.



aabb is accepted along path : $0 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

BUT... it is not accepted along the valid path:

$0 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 0$



Deterministic Finite Automata

A DFA is an NFA with the following restrictions:

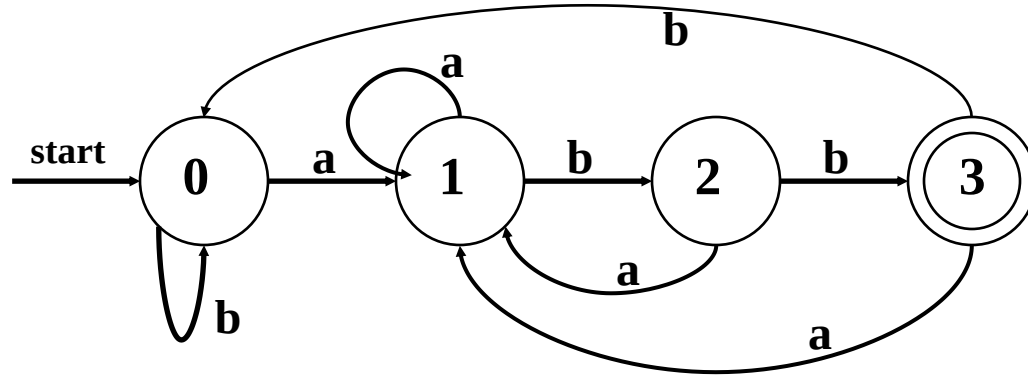
- ϵ moves are not allowed
- For every state $s \in S$, there is one and only one path from s for every input symbol $a \in \Sigma$.

Since transition tables don't have any alternative options, DFAs are easily simulated via an algorithm.

```
s ← s0
c ← nextchar;
while c ≠ eof do
    s ← move(s, c);
    c ← nextchar;
end;
if s is in F then return "yes"
else return "no"
```

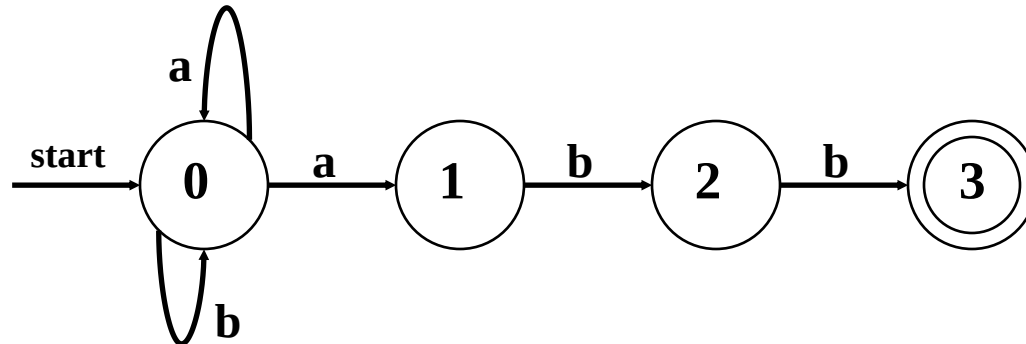


Example – DFA : $(a \mid b)^*abb$



DFA

Recall the original NFA:



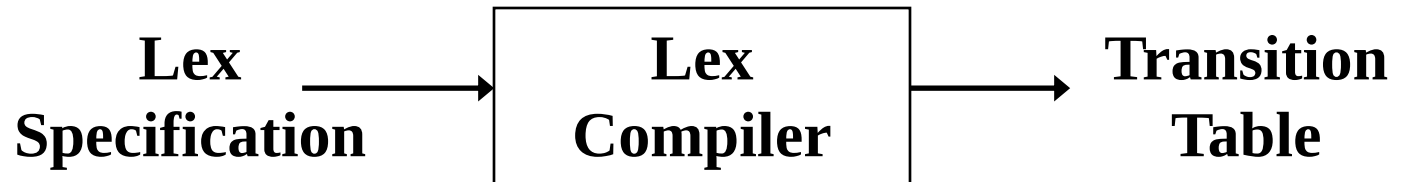
Relation between RE, NFA and DFA

1. There is an algorithm for converting any RE into an NFA.
2. There is an algorithm for converting any NFA to a DFA.
3. There is an algorithm for converting any DFA to a RE.

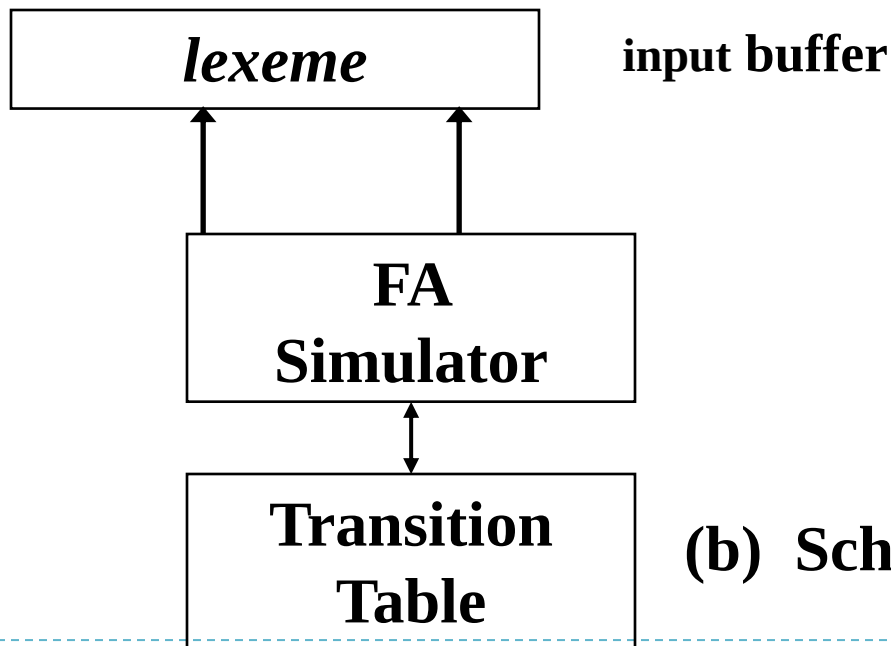
These facts tell us that **REs, NFAs and DFAs have equivalent expressive power.**

All three describe the class of **regular languages.**

Pictorially



(a) Lex Compiler



(b) Schematic lexical analyzer

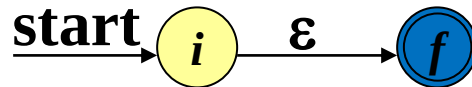
Converting Regular Expressions to NFAs

- How do we define an NFA that accepts a regular expression?
- It is very simple. Remember that a regular expression is formed by the use of alternation, concatenation, and repetition.
- Thus all we need to do is to know how to build the NFA for a single symbol, and how to compose NFAs.

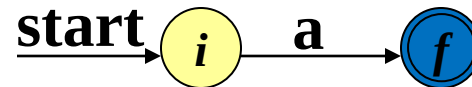
Converting Regular Expressions to NFAs

Thompson's Construction

- ▶ Empty string ε is a regular expression denoting $\{\varepsilon\}$



- ▶ a is a regular expression denoting $\{a\}$ for any a in Σ

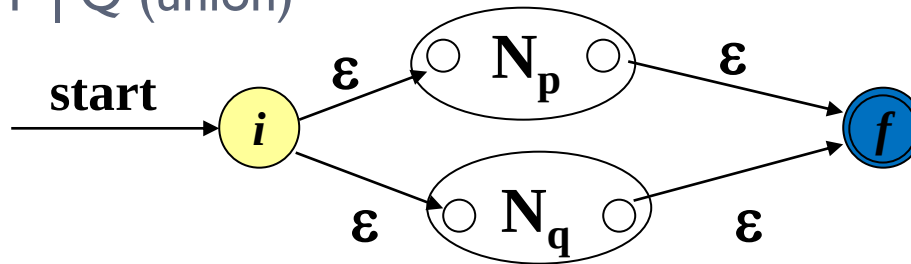


Converting Regular Expressions to NFAs

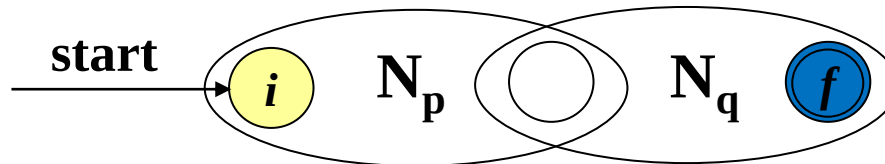
Composing NFAs with Alternation and Concatenation

If P and Q are regular expressions with NFAs N_p, N_q :

$P \mid Q$ (union)



PQ (concatenation)

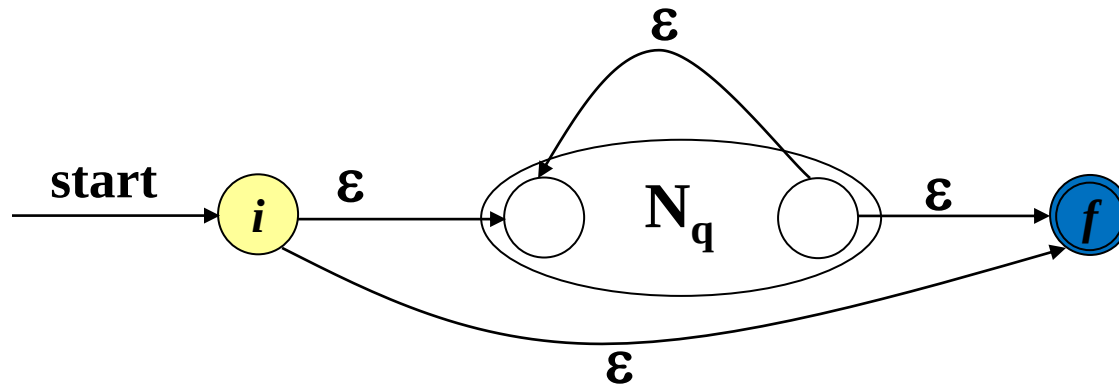


Converting Regular Expressions to NFAs

Composing NFAs with Repeation

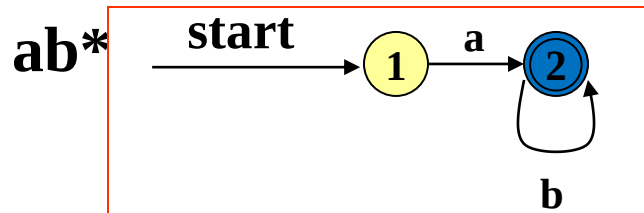
If Q is a regular expression with NFA N_q :

Q^* (closure)

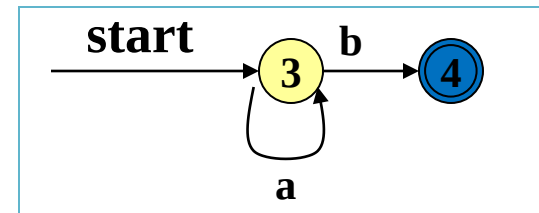


Example $(ab^* \mid a^*b)^*$

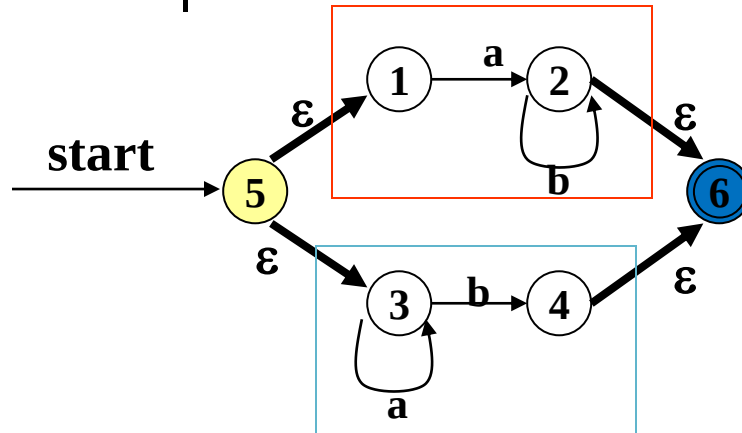
Starting with:



a^*b

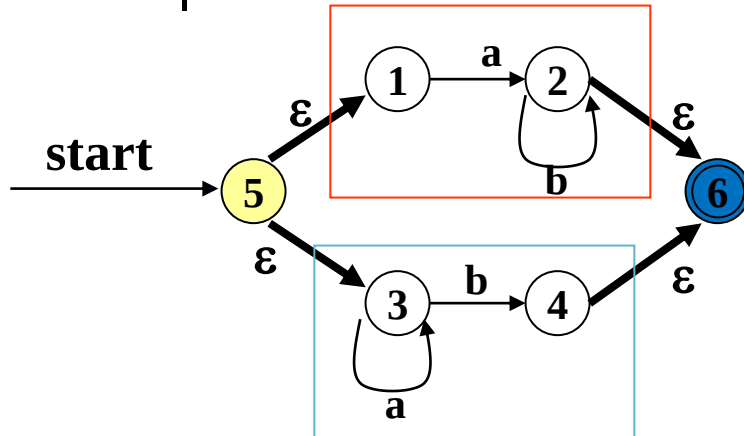


$ab^* \mid a^*b$

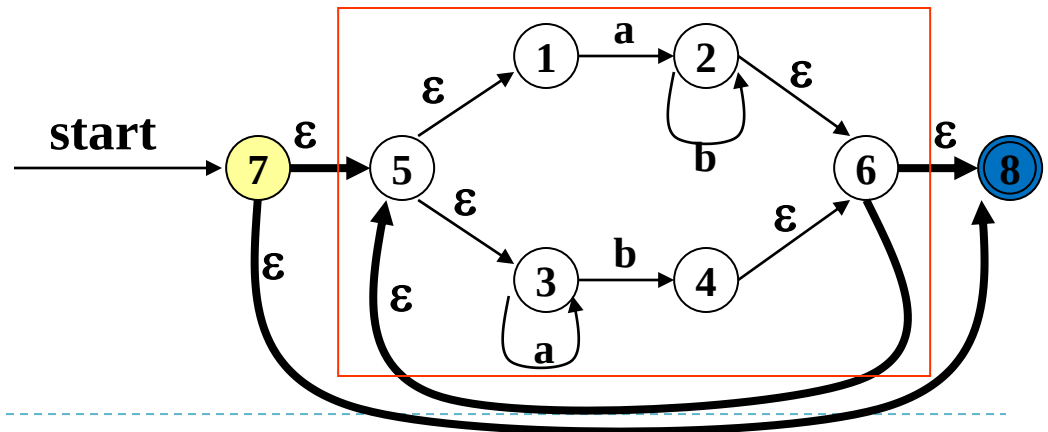


Example $(ab^* \mid a^*b)^*$

$ab^* \mid a^*b$



$(ab^* \mid a^*b)^*$



Properties of Construction

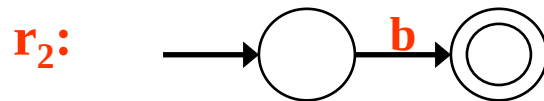
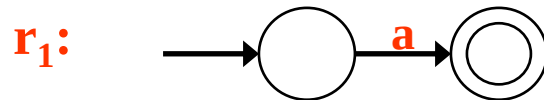
Let r be a regular expression, with NFA $N(r)$, then

1. $N(r)$ has #of states $\leq 2^{*}(\text{\#symbols} + \text{\#operators})$ of r
2. $N(r)$ has exactly one start and one accepting state
3. Each state of $N(r)$ has at most one outgoing edge $a \in \Sigma$ or at most two outgoing ϵ -transition

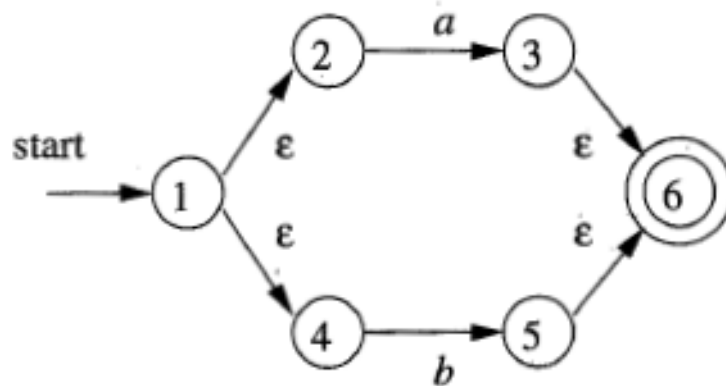


Detailed Example – NFA Construction(1)

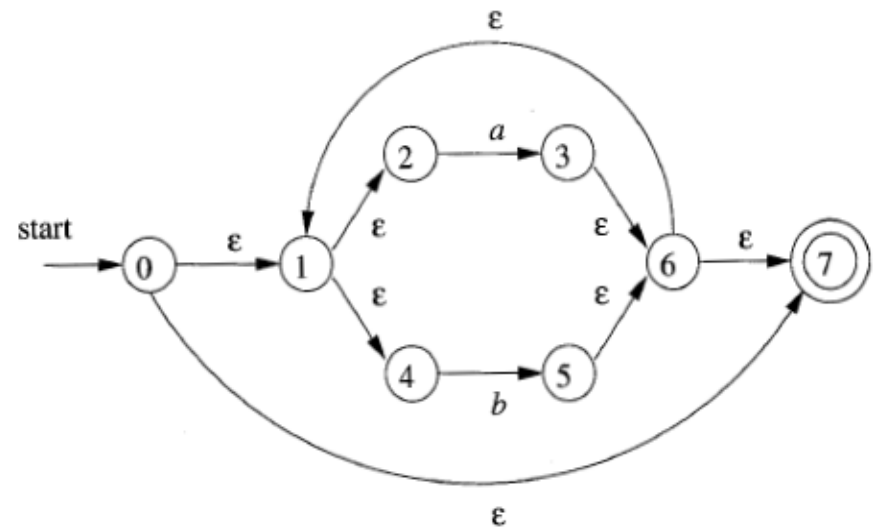
$(a|b)^*abb$



$r_3: r_1 | r_2$



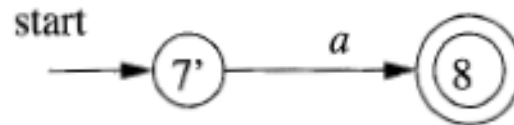
$r_4: (r_3)^*$



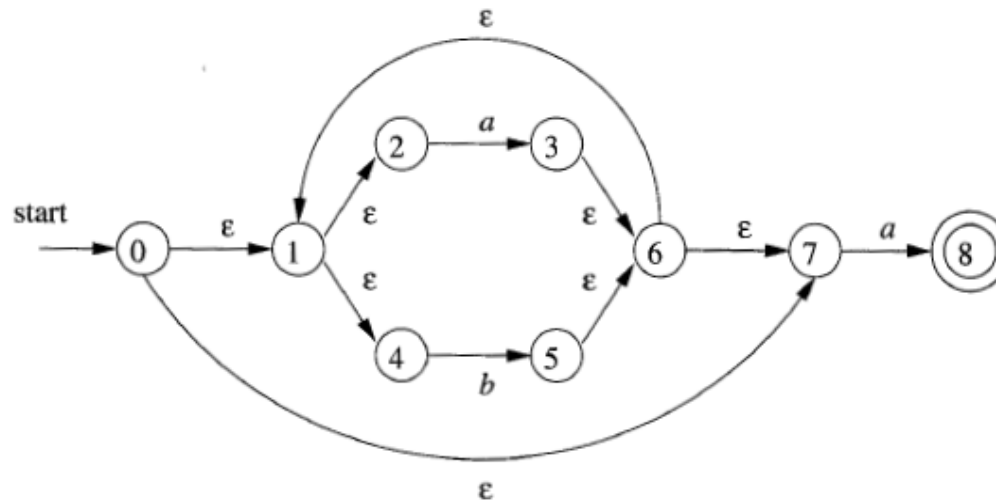
Detailed Example – NFA Construction(2)

$(a|b)^*abb$

r_5



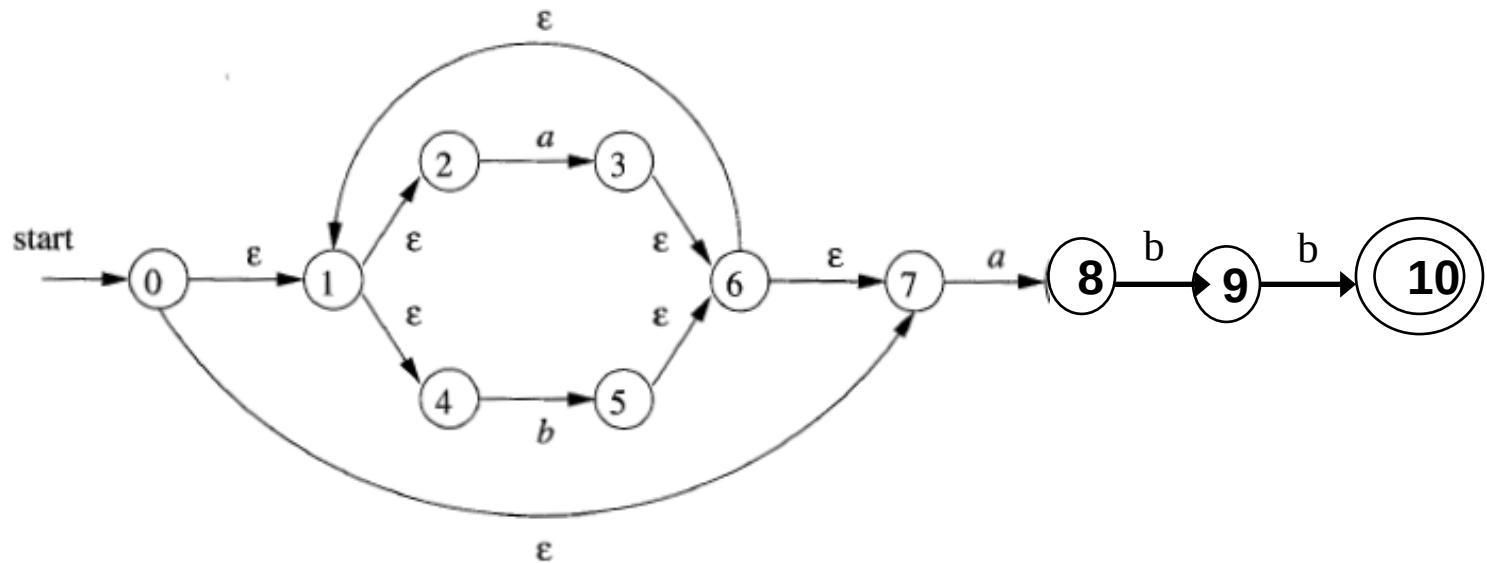
$r_6: r_4 \ r_5$



Detailed Example – Final Step

$(a|b)^*abb$

r_{10}



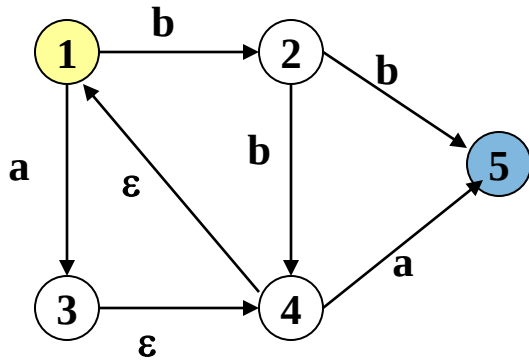
Converting NFAs to DFAs

(subset construction)

- ▶ **Idea:** Each state in the new DFA will correspond to some set of states from the NFA. The DFA will be in state $\{s_0, s_1, \dots\}$ after input if the NFA could be in *any* of these states for the same input.
- ▶ **Input:** NFA N with state set S_N , alphabet S , start state s_N , final states F_N , transition function $T_N: S_N \times \{S \cup \epsilon\} \rightarrow S_N$
- ▶ **Output:** DFA D with state set S_D , alphabet S , start state $s_D = \epsilon\text{-closure}(s_N)$, final states F_D , transition function $T_D: S_D \times S \rightarrow S_D$

Terminology: ϵ -closure

ϵ -closure(T) = T + all NFA states reachable from any state in T using only ϵ transitions.



ϵ -closure({1,2,5}) = {1,2,5}

ϵ -closure({4}) = {1,4}

ϵ -closure({3}) = {1,3,4}

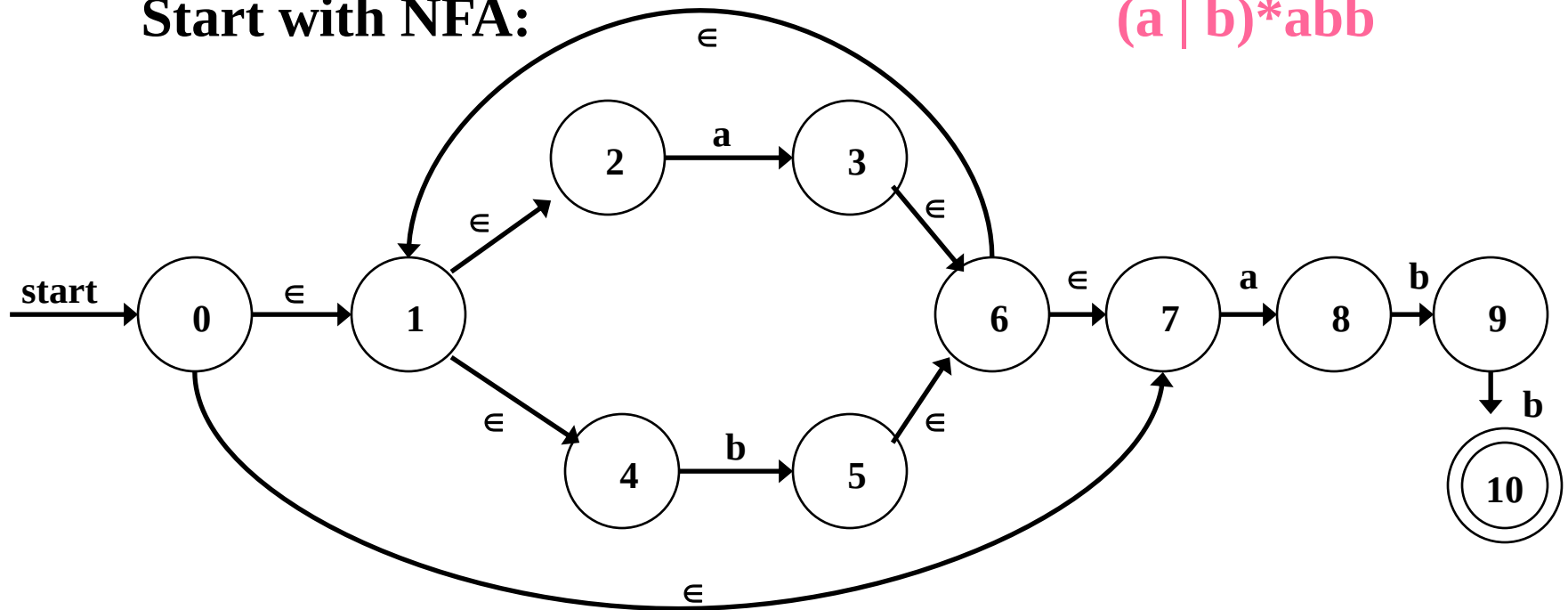
ϵ -closure({3,5}) = {1,3,4,5}



Illustrating Conversion – An Example

Start with NFA:

$(a \mid b)^*abb$



First we calculate: ϵ -closure(0) (i.e., state 0)

ϵ -closure(0) = {0, 1, 2, 4, 7} (all states reachable from 0 on ϵ -moves)

Let $A = \{0, 1, 2, 4, 7\}$ be a state of new DFA, D.

Conversion Example – continued (1)

2nd, we calculate : $a : \epsilon\text{-closure}(\text{move}(A,a))$ and
 $b : \epsilon\text{-closure}(\text{move}(A,b))$

a : $\epsilon\text{-closure}(\text{move}(A,a)) = \epsilon\text{-closure}(\text{move}(\{0,1,2,4,7\},a))$
adds $\{3,8\}$ (since $\text{move}(2,a)=3$ and $\text{move}(7,a)=8$)

From this we have : $\epsilon\text{-closure}(\{3,8\}) = \{1,2,3,4,6,7,8\}$
(since $3 \rightarrow 6 \rightarrow 1 \rightarrow 4$, $6 \rightarrow 7$, and $1 \rightarrow 2$ all by ϵ -moves)

Let $B = \{1,2,3,4,6,7,8\}$ be a new state. Define **Dtran[A,a] = B**.

b : $\epsilon\text{-closure}(\text{move}(A,b)) = \epsilon\text{-closure}(\text{move}(\{0,1,2,4,7\},b))$
adds $\{5\}$ (since $\text{move}(4,b)=5$)

From this we have : $\epsilon\text{-closure}(\{5\}) = \{1,2,4,5,6,7\}$
(since $5 \rightarrow 6 \rightarrow 1 \rightarrow 4$, $6 \rightarrow 7$, and $1 \rightarrow 2$ all by ϵ -moves)

Let $C = \{1,2,4,5,6,7\}$ be a new state. Define **Dtran[A,b] = C**.

Conversion Example – continued (2)

3rd, we calculate for state B on {a,b}

$$\underline{a} : \epsilon\text{-closure}(\text{move}(B,a)) = \epsilon\text{-closure}(\text{move}(\{1,2,3,4,6,7,8\},a)) \\ = \{1,2,3,4,6,7,8\} = B$$

Define $\text{Dtran}[B,a] = B$.

$$\underline{b} : \epsilon\text{-closure}(\text{move}(B,b)) = \epsilon\text{-closure}(\text{move}(\{1,2,3,4,6,7,8\},b)) \\ = \{1,2,4,5,6,7,9\} = D$$

Define $\text{Dtran}[B,b] = D$.

4th, we calculate for state C on {a,b}

$$\underline{a} : \epsilon\text{-closure}(\text{move}(C,a)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7\},a)) \\ = \{1,2,3,4,6,7,8\} = B$$

Define $\text{Dtran}[C,a] = B$.

$$\underline{b} : \epsilon\text{-closure}(\text{move}(C,b)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7\},b)) \\ = \{1,2,4,5,6,7\} = C$$

Define $\text{Dtran}[C,b] = C$.

Conversion Example – continued (3)

5th, we calculate for state D on {a,b}

$$\underline{a} : \epsilon\text{-closure}(\text{move}(D,a)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7,9\},a)) \\ = \{1,2,3,4,6,7,8\} = B$$

Define $D\text{tran}[D,a] = B$.

$$\underline{b} : \epsilon\text{-closure}(\text{move}(D,b)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7,9\},b)) \\ = \{1,2,4,5,6,7,10\} = E$$

Define $D\text{tran}[D,b] = E$.

Finally, we calculate for state E on {a,b}

$$\underline{a} : \epsilon\text{-closure}(\text{move}(E,a)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7,10\},a)) \\ = \{1,2,3,4,6,7,8\} = B$$

Define $D\text{tran}[E,a] = B$.

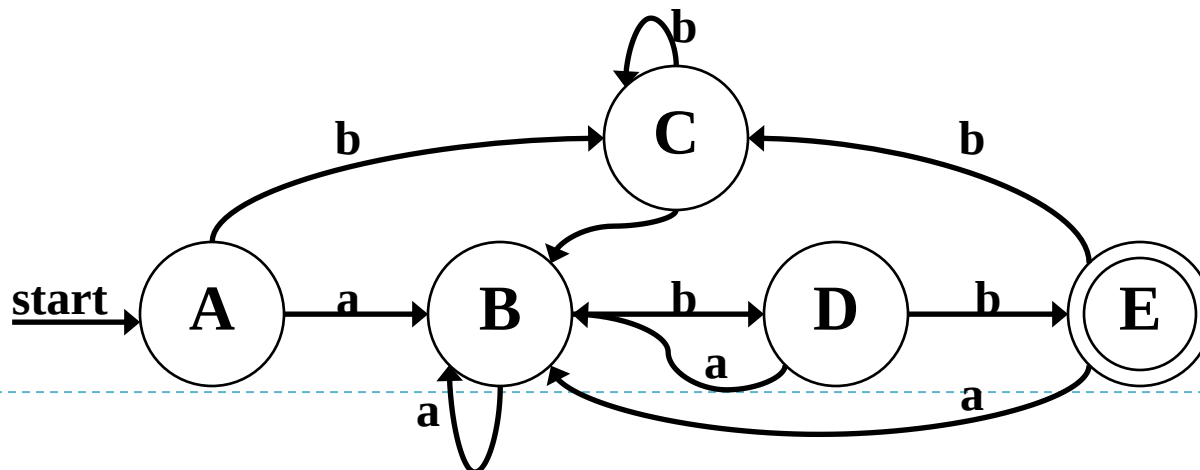
$$\underline{b} : \epsilon\text{-closure}(\text{move}(E,b)) = \epsilon\text{-closure}(\text{move}(\{1,2,4,5,6,7,10\},b)) \\ = \{1,2,4,5,6,7\} = C$$

Define $D\text{tran}[E,b] = C$.

Conversion Example – continued (4)

This gives the transition table **Dtran** for the DFA of:

Dstates	Input Symbol	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



Algorithm For Subset Construction

push all states in T onto stack;

initialize ϵ -closure(T) to T ;

while stack is not empty **do begin**

 pop t , the top element, off the stack;

for each state u with edge from t to u labeled ϵ **do**

if u is not in ϵ -closure(T) **do begin**

 add u to ϵ -closure(T) ;

 push u onto stack

end

end

**computing the
 ϵ -closure**



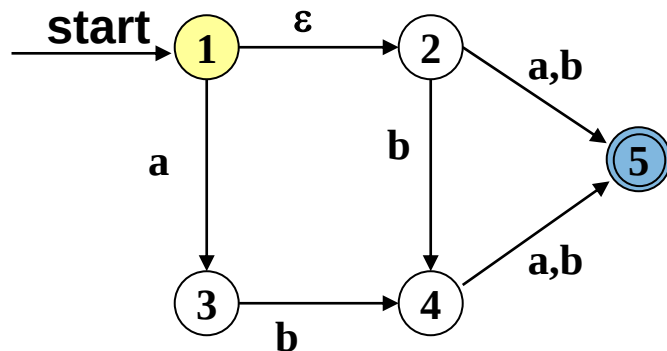
Algorithm For Subset Construction – (2)

```
initially,  $\epsilon$ -closure( $s_0$ ) is only (unmarked) state in Dstates;  
while there is unmarked state T in Dstates do begin  
    mark T;  
    for each input symbol  $a$  do begin  
         $U := \epsilon$ -closure(move(T, $a$ ));  
        if U is not in Dstates then  
            add U as an unmarked state to Dstates;  
            Dtran[T, $a$ ] := U  
        end  
    end  
end
```



Example 2: Subset Construction

NFA



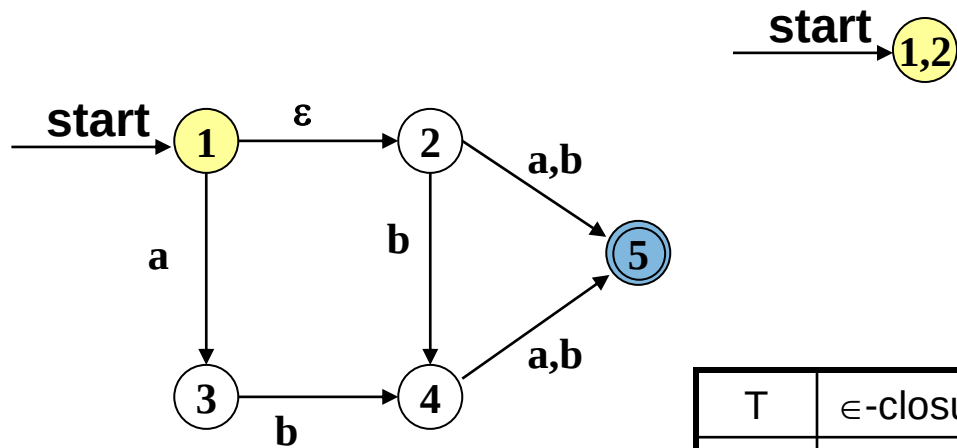
NFA N with

- State set $S_N = \{1,2,3,4,5\}$,
- Alphabet $\Sigma = \{a,b\}$
- Start state $s_N=1$,
- Final states $F_N=\{5\}$,
- Transition function $T_N: S_N \times \{\Sigma \cup \varepsilon\} \rightarrow S_N$

	a	b	ε
1	3	-	2
2	5	5, 4	-
3	-	4	-
4	5	5	-
5	-	-	-

Example 2: Subset Construction

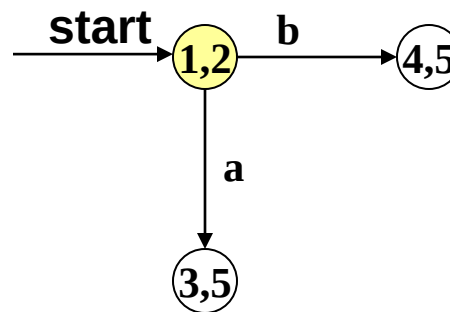
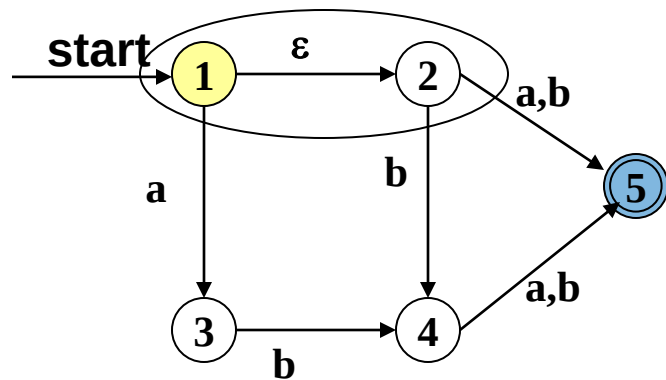
NFA



T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}		

Example 2: Subset Construction

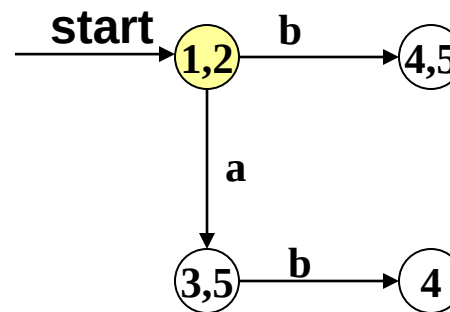
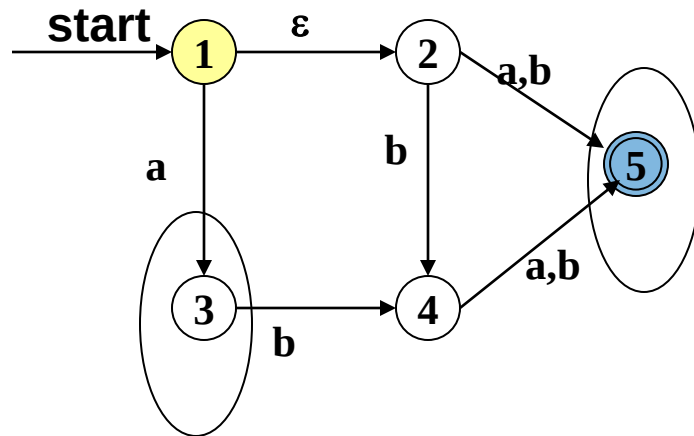
NFA



T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}	{3,5}	{4,5}
{3,5}		
{4,5}		

Example 2: Subset Construction

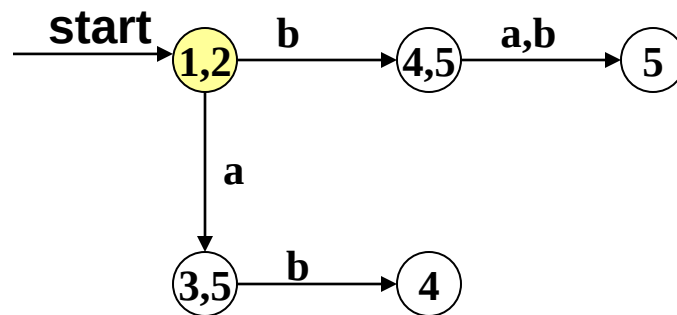
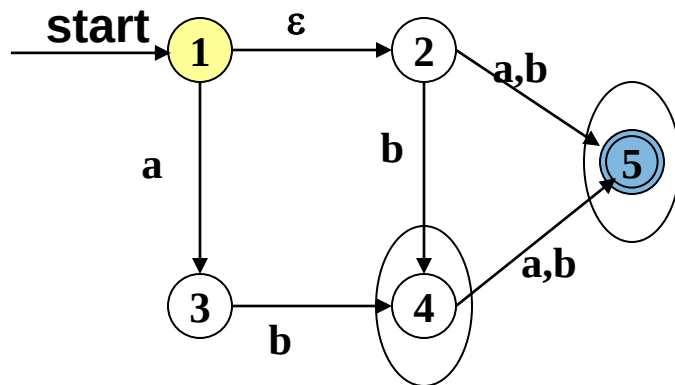
NFA



T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}		
{4}		

Example 2: Subset Construction

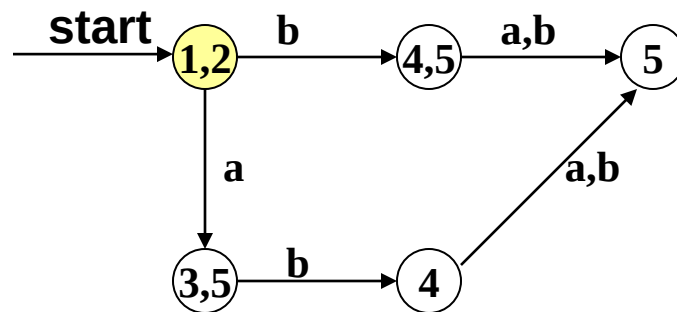
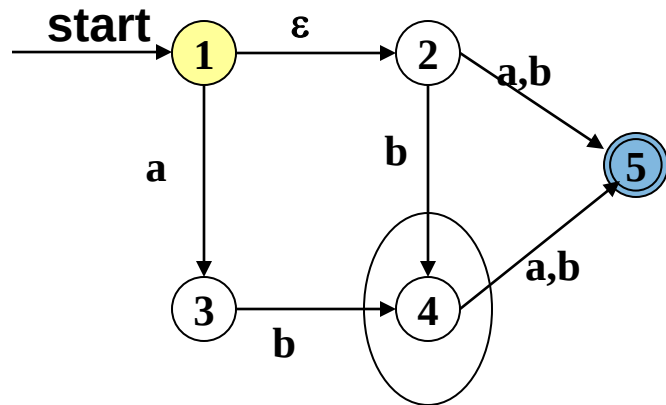
NFA



T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}	{5}	{5}
{4}		
{5}		

Example 2: Subset Construction

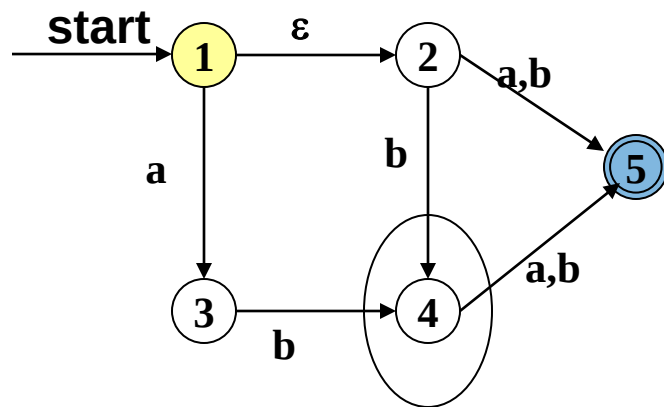
NFA



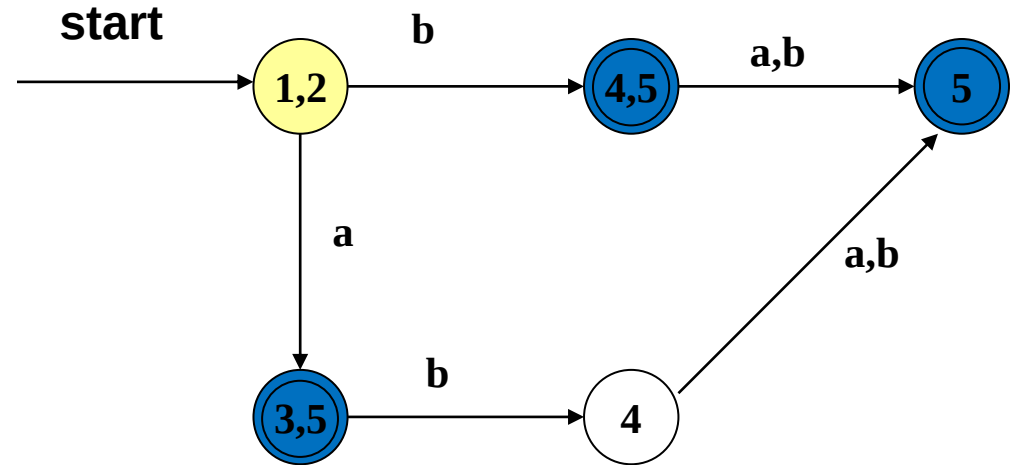
T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}	{5}	{5}
{4}	{5}	{5}
{5}	-	-

Example 2: Subset Construction

NFA



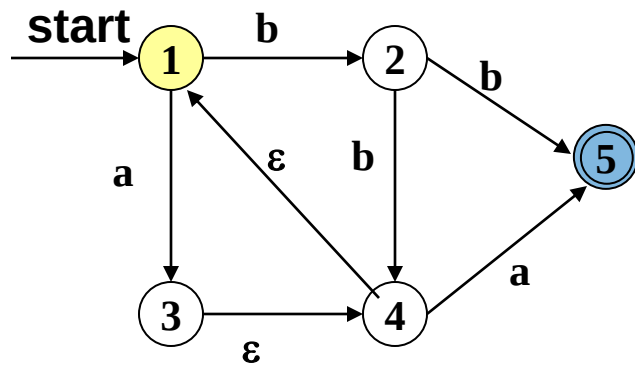
All final states since the NFA final state is included



T	ϵ -closure(move(T, a))	ϵ -closure(move(T, b))
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}	{5}	{5}
{4}	{5}	{5}
{5}	-	-

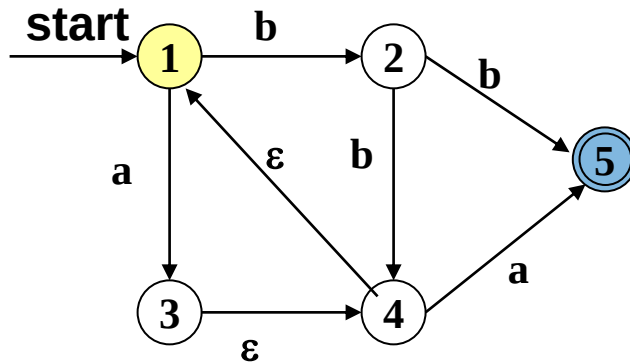
Example 3: Subset Construction

NFA

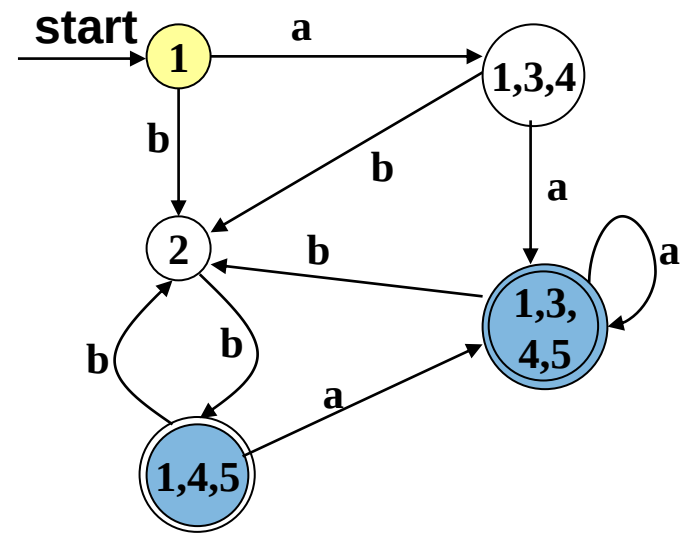


Example 3: Subset Construction

NFA

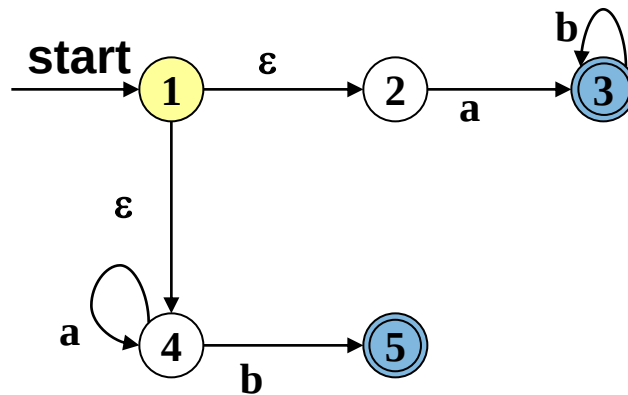


DFA

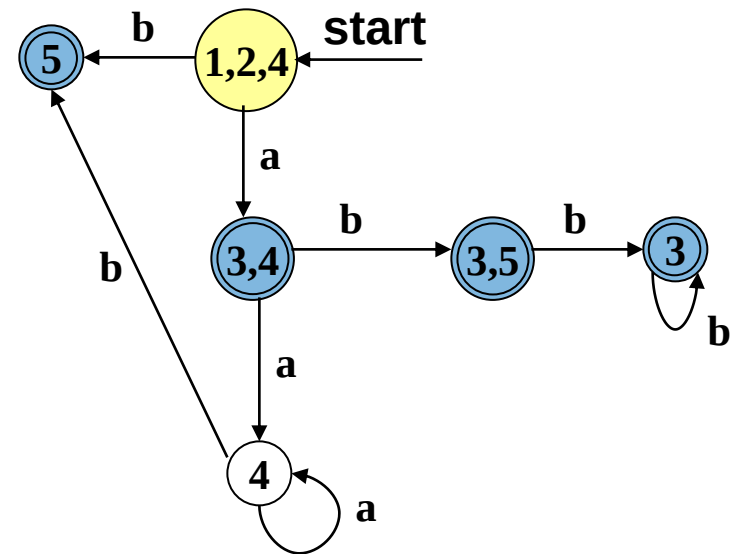


Example 4: Subset Construction

NFA

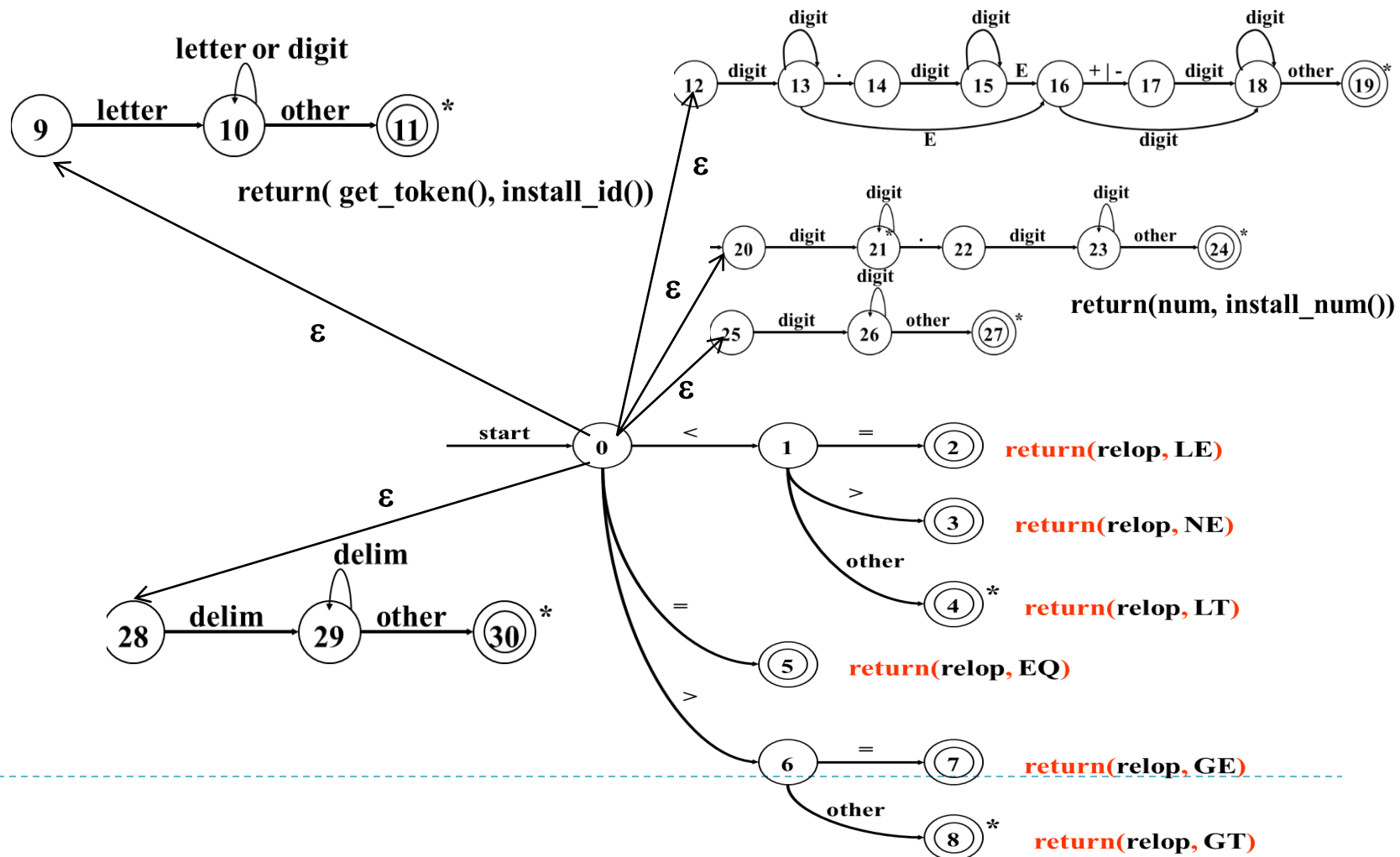


DFA



NFA: Transition Diagram capturing Multiple Tokens

Questions: Draw the transition diagram that will recognize **id**, **keywords**, **number**, **whitespaces** and **relational operators**?



Corresponding DFA: Transition Diagram capturing Multiple Tokens

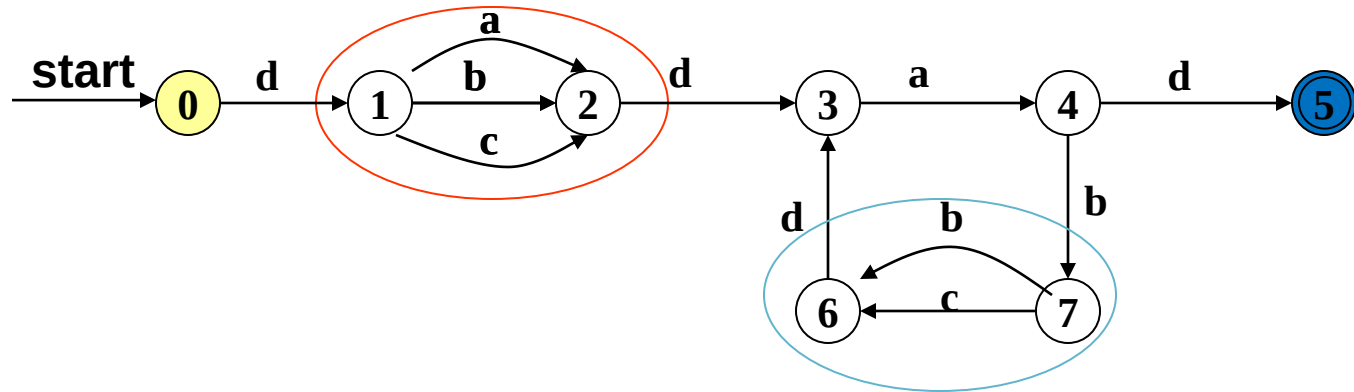
Questions: Draw the transition diagram that will recognize id, keywords, number, whitespaces and relational operators?



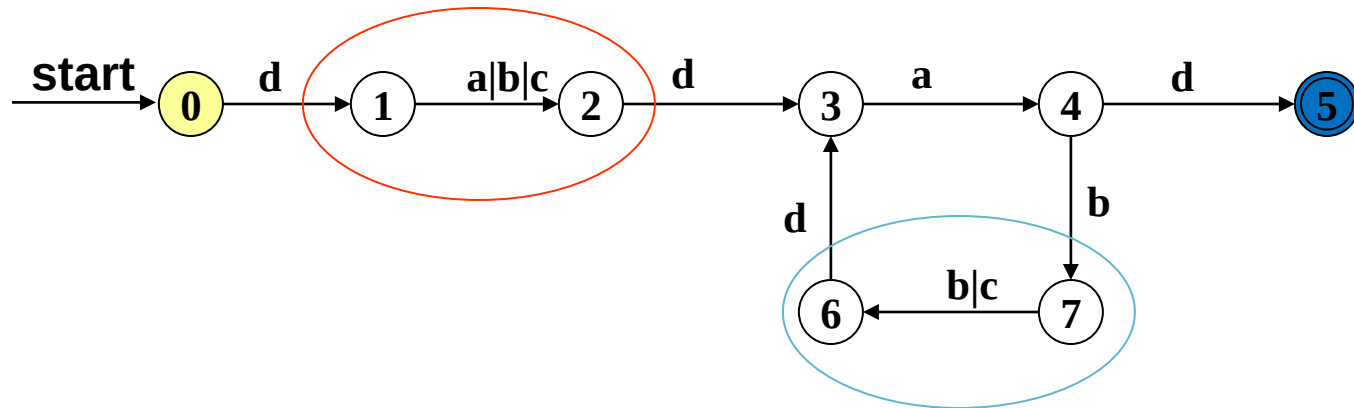
Converting DFAs to REs

1. Combine serial links by concatenation
2. Combine parallel links by alternation
3. Remove self-loops by Kleene closure
4. Select a node (other than initial or final) for removal. Replace it with a set of equivalent links whose path expressions correspond to the in and out links
5. Repeat steps 1-4 until the graph consists of a single link between the entry and exit nodes.

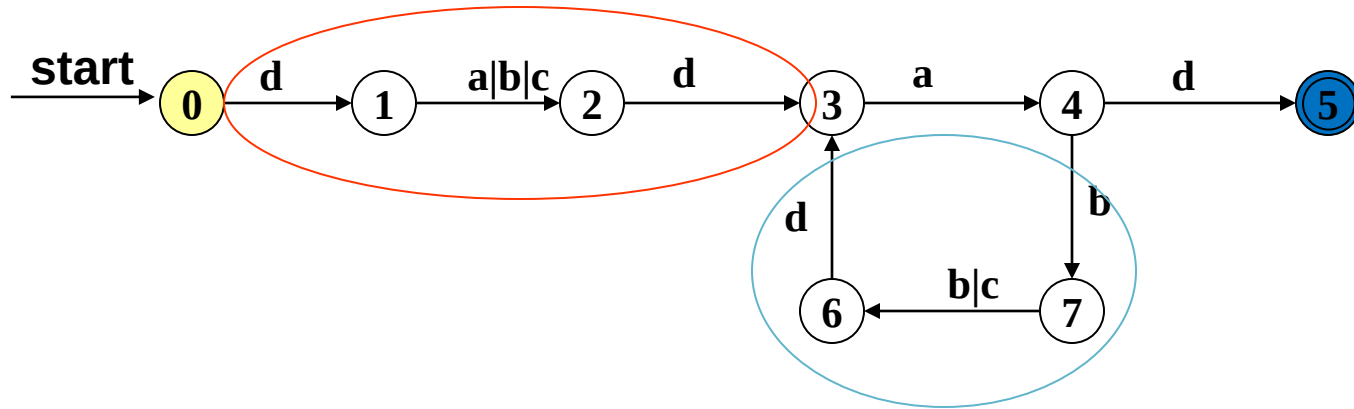
Example: DFAs to REs



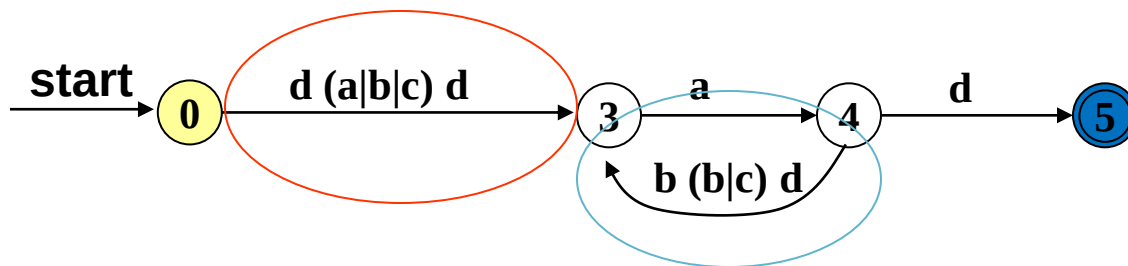
parallel edges become alternation



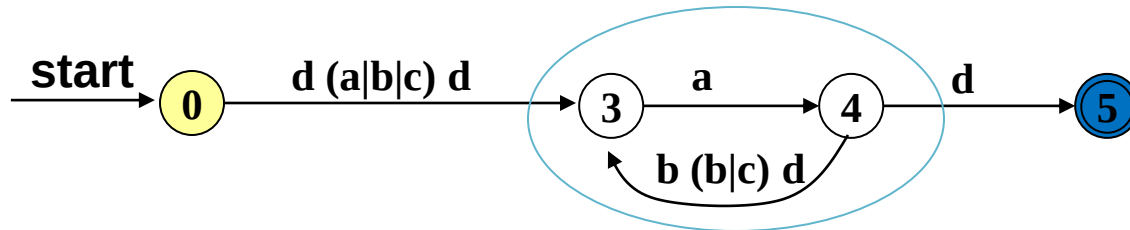
Example: DFAs to REs



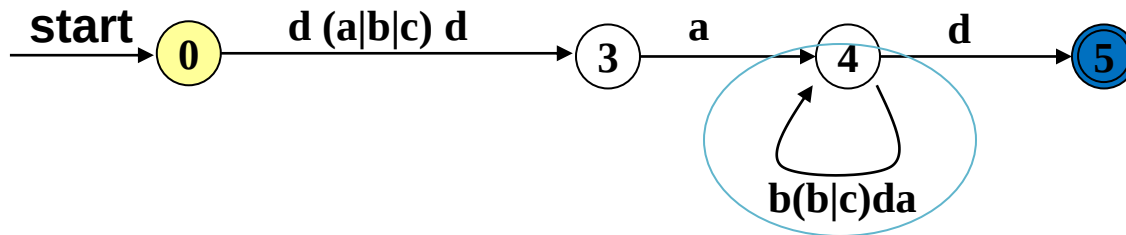
serial edges become concatenation



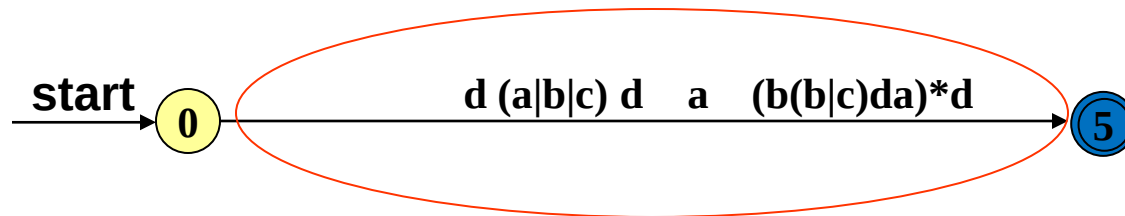
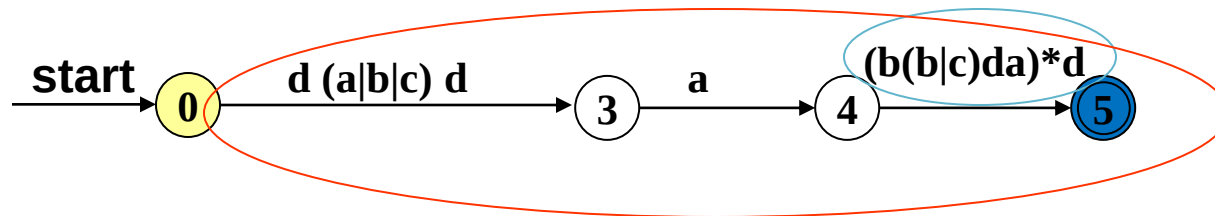
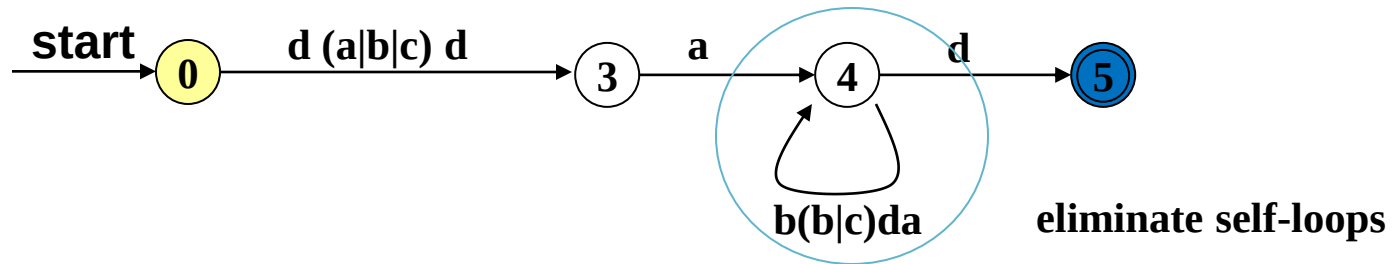
Example: DFAs to REs



Find paths that can be “shortened”



Example: DFAs to REs



Exercises for Practice

Dragon Book (Chapter-3):

3.3.5, 3.4.1, 3.4.2, 3.6.2, 3.6.3, 3.6.4, 3.6.5,
3.7.1, 3.7.2, 3.7.3

